

算法总结

$$T(n) = \begin{cases} c & n = 1 \\ aT(n/b) + cn^k & n > 1 \end{cases}$$

$$T(n) = \begin{cases} O(n^{\log_b a}) & a > b^k \\ O(n^k \log_b n) & a = b^k \\ O(n^k) & a < b^k \end{cases}$$

分治

分治原理

伪代码

时间复杂度

最优:

最差:

平均:

递归方程

归并排序

分治原理

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

伪代码

```

1 void merge(int* nums, int left, int mid, int right) {
2     int* b = new int[N];
3     int i, j, k;
4     for (int u = left; u <= right; u++) b[u] = nums[u];
5     for (i = left, j = mid + 1, k = i; i <= mid && j <= right; k++) {
6         if (b[i] <= b[j]) {
7             nums[k] = b[i++];
8         }
9         else {
10            nums[k] = b[j++];
11        }
12    }
13    while (i <= mid) {
14        nums[k++] = b[i++];
15    }
16    while (j <= right) {
17        nums[k++] = b[j++];
18    }
19 }
20
21 void mergesort(int* nums, int left, int right) {
22     if (left < right) {
23         int mid = (left + right) / 2;
24         mergesort(nums, left, mid);
25         mergesort(nums, mid + 1, right);
26         merge(nums, left, mid, right);
27     }
28 }

```

时间复杂度

最优: $O(n \log n)$

最差: $O(n \log n)$

平均: $O(n \log n)$

递归方程

复杂度分析

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

二分

分治原理:

取升序数组中点k, 若待搜索数据大于中点数据, 则递归到右半边, 否则递归到左半边

伪代码:

//分治法：二分搜索

```
public static int binarySearch(int [] a, int x, int n)
{
    // 在 a[0] <= a[1] <= ... <= a[n-1] 中搜索 x
    // 找到x时返回其在数组中的位置，否则返回-1
    int left = 0; int right = n - 1;
    while (left <= right)
    {
        int middle = (left + right)/2;
        if (x == a[middle]) return middle;
        if (x > a[middle]) left = middle + 1; //在右半部查找
        else right = middle - 1;           //在左半部查找
    }
    return -1; // 未找到x
}
```

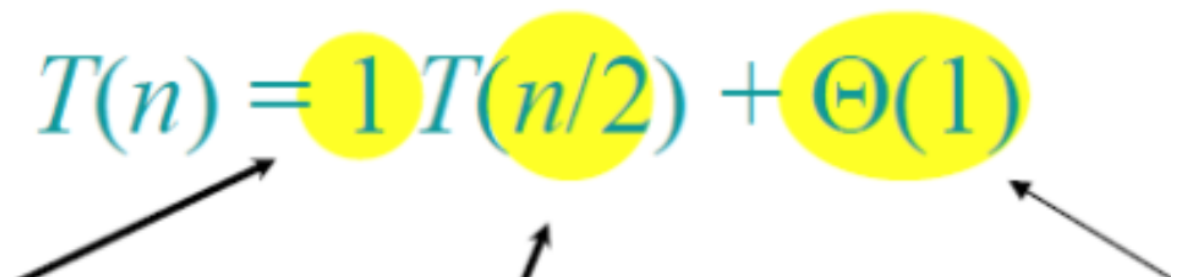
时间复杂度

最优: $O(1)$

最差: $O(\log n)$

平均: $O(\log n)$

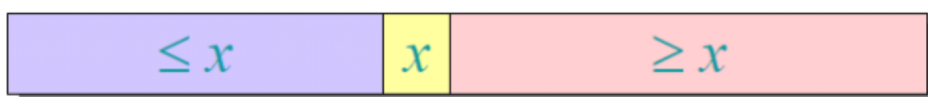
递归方程

$$T(n) = 1T(n/2) + \Theta(1)$$
A diagram showing the recurrence equation $T(n) = 1T(n/2) + \Theta(1)$. The terms '1', 'T(n/2)', and ' $\Theta(1)$ ' are each enclosed in a yellow circle. Three arrows point to these circles: one from the bottom-left to the '1' circle, one from the bottom-center to the 'T(n/2)' circle, and one from the bottom-right to the ' $\Theta(1)$ ' circle.

快速排序

分治原理

- **分解**：数组 $A[p \dots r]$ 被划分成两个子数组 $A[p..q-1]$ 和 $A[q+1 \dots r]$, 且 $A[q]$ 为下标（标杆）元素。使得
 - $A[p..q-1]$ 都 $\leq A[q]$, $A[q+1 \dots r] > A[q]$



- **解决**：递归地调用快速排序，对子数组 $A[p..q-1]$ 和 $A[q+1 \dots r]$, 排序

伪代码

```
1  int partition(int* nums, int left, int right) {
2      int key = nums[left];
3      while (left < right) {
4          if (nums[right] >= key && left < right) right--;
5          nums[left] = nums[right];
6          if (nums[left] <= key && left < right) left++;
7          nums[right] = nums[left];
8      }
9      nums[left] = key;
10     return left;
11 }
12
13 void quicksort(int* nums, int left, int right) {
14     if (left < right) {
15         int pos = partition(nums, left, right);
16         quicksort(nums, left, pos - 1);
17         quicksort(nums, pos + 1, right);
18     }
19 }
```

时间复杂度

最优: $O(n \log n)$

最差: $O(n^2)$

平均: $O(n \log n)$

递归方程

$$L(n) = 2U(n/2) + \Theta(n) \quad \text{*lucky*}$$
$$U(n) = L(n-1) + \Theta(n) \quad \text{*unlucky*}$$

线性时间选择

分治原理

如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ϵ 倍($0 < \epsilon < 1$ 是某个正常数)，那么就可以在最坏情况下用 $O(n)$ 时间完成选择任务，**这是线性时间选择问题。**

1. 将 n 个输入元素划分成 $n/5$ 个组，每组5个元素。用任何一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $n/5$ 个。
2. 递归调用select来找出这 $n/5$ 个元素的中位数。如果 $n/5$ 是偶数，就找它的2个中位数中较大的一个。
3. 以这个元素作为划分基准。

伪代码

Type **Select**(Type a[], int p, int r, int k)

{

if (r-p<75) {

用某个简单排序算法对数组a[p:r]排序;

return a[p+k-1];

};

for (int i = 0; i <= (r-p-4)/5; i++)

*将a[p+5*i] 至 a[p+5*i+4]的第3小元素
与a[p+i]交换位置;*

//找中位数的中位数， r-p-4即上面所说的n-5

Type x = Select(a, p, p+(r-p-4)/5, (r-p-4)/10);

int i=Partition(a, p, r, x),

j=i-p+1;

if (k<=j) return Select(a, p, i, k);

else return Select(a, i+1, r, k-j);

}

时间复杂度

最优: $O(n)$

最差: $O(n)$

平均: $O(n)$

递归方程

复杂度分析

$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n) = O(n)$$

最近点对

分治原理

分治法的思路:

- ① 划分成子规模点集S1和S2
- ② 找到S1和S2的最小距离d。
- ③ 合并S1和S2: 并利用d在S1的(m-d,m]和S2的(m,m+d]中找最大点和最小点, 即p3和q3。选出最小距离, 完成合并

伪代码

//一维

```
bool Cpair1( s, d )
```

```
{
```

```
    n = |s|;
```

```
    if (n<2) {d为无穷大; return false;}
```

```
    m = s中各个坐标的中位数;
```

```
    //构造S1 和 S2
```

```
    //S1 = { x<m} s2 = {x > m }
```

```
    Cpair1( s1, d1 );
```

```
    Cpair1( s2, d2 );
```

```
    p = max (s1);
```

```
    q = min (s2);
```

```
    d = min( d1, d2, q-p);
```

```
    return true;
```

```
}
```

有时候明明很累

却不知该为什么而坚持

学习8

```
double cpair2(S)
```

```
{
```

```
    n=|S|;
```

```
    if (n < 2) return false;
```

```
1、 m=S中各点x间坐标的中位数;
```

```
    构造S1和S2;
```

```
        //S1={p∈S|x(p)≤m},
```

```
        //S2={p∈S|x(p)>m}
```

```
2、 d1=cpair2(S1);
```

```
    d2=cpair2(S2);
```

```
3、 dm=min(d1,d2);
```

4、 设P1是S1中距垂直分割线l的距离在dm之内的所有点组成的集合;

P2是S2中距分割线l的距离在dm之内所有点组成的集合;

将P1和P2中点依其y坐标值排序;

并设X和Y是相应的已排好序的点列;

5、 通过扫描X以及对于X中每个点检查Y中与其距离在dm之内的所有点(最多6个)可以完成合并;

当X中的扫描指针逐次向上移动时, Y中的扫描指针可在宽为2dm的区间内移动;

设dl是按这种扫描方式找到的点对间的最小距离;

6、 d=min(dm,dl);

```
    return d;
```

```
}
```

时间复杂度

最优: $O(n \log n)$

最差: $O(n \log n)$

平均: $O(n \log n)$

递归方程

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$T(n) = O(n \log n)$$

动态规划

递归方程式

伪代码

复杂度

最优:

最差:

平均:

矩阵连乘

递归方程式

- 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i,j]$, 则原问题的最优值为 $m[1,n]$
- 当 $i=j$ 时, $A[i:j]=A_i$, 因此, $m[i,i]=0$, $i=1,2,\dots,n$
- 当 $i < j$ 时, $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$

这里 A_i 的维数为 $p_{i-1} \times p_i$

- 由最优子结构性质, 可以递归地定义 $m[i,j]$ 为:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

k 的位置只有 $j-i$ 种可能

伪代码

```

void MatrixChain(int *p, int n, int **m, int **s)
{
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            m[i][j] = m[i + 1][j] + p[i - 1] * p[i] * p[j];
            s[i][j] = i;
            for (int k = i + 1; k < j; k++) {
                int t = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
            }
        }
    }
}

```

复杂度

最优: $O(n^3)$

最差: $O(n^3)$

平均: $O(n^3)$

最长公共子序列

递归方程式

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

伪代码

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b)
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0; /*初始化, Y[j]为空时
    for (i = 1; i <= n; i++) c[0][i] = 0; /*初始化, X[i]为空时

    for (i = 1; i <= m; i++) /*两重循环, 自下而上, 计算子问题 {X(i), Y(j)}
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j]) { /*情况 1
                c[i][j]=c[i-1][j-1]+1;
                b[i][j]=1; }
            else if (c[i-1][j]>=c[i][j-1]) { /*情况 2
                c[i][j]=c[i-1][j]; b[i][j]=2; }
            else {
                c[i][j]=c[i][j-1]; /*情况 3
                b[i][j]=3; }
        }
    }
```

```

void LCS(int i, int j, char *x, int **b)
{
    if (i == 0 || j == 0) return;
    if (b[i][j] == 1) { LCS(i-1, j-1, x, b); cout << x[i]; }
    else if (b[i][j] == 2) LCS(i-1, j, x, b);
    else LCS(i, j-1, x, b);
}

```

复杂度

最优: $O(mn)$

最差: $O(mn)$

平均: $O(mn)$

最大字段和

递归方程式

记 $b[j] = \max_{1 \leq i \leq j} \{ \sum_{k=i}^j a[k] \}$, $1 \leq j \leq n$, 则所求的最大子段和为

$$\max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} \max_{1 \leq i \leq j} \sum_{k=i}^j a[k] = \max_{1 \leq j \leq n} b[j]$$

当 $b[j-1] > 0$ 时 $b[j] = b[j-1] + a[j]$, 否则 $b[j] = a[j]$ 。由此可得计算 $b[j]$ 的动态规划递归式

$$b[j] = \max\{b[j-1] + a[j], a[j]\}, \quad 1 \leq j \leq n$$

```
void getMax(int a[],int n) {  
    int dp, tmp;  
    dp=Max=a[n-1];  
    tmp=n-1;  
    B=n-1;  
    E=n-1;  
    for(int i=n-2;i>=0;i--) {  
        if(dp>0)  
            dp=dp+a[i];  
        else {  
            dp=a[i];  
            tmp=i;  
        }  
        if(Max<=dp) {  
            B=i;  
            E=tmp;  
            Max=dp;  
        }  
    }  
}
```

复杂度

最优: $O(n)$

最差: $O(n)$

平均: $O(n)$

01背包

递归方程式

的最优值为 $m(i, j)$ ，即 $m(i, j)$ 是背包容量为 j ，可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质，可以建立计算 $m(i, j)$ 的递归式如下。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

伪代码

```
1  int bag01() {
2      for (int i = 1; i <= n; i++) {
3          for (int j = v; j >= cost[i]; j--) {
4              int temp = dp[j - cost[i]] + value[i];
5              dp[j] = max(dp[j], dp[j - cost[i]] + value[i]);
6              if (dp[j] == temp) {
7                  flag[i][j] = i;
8              }
9          }
10     }
11     return dp[v];
12 }
```

复杂度

最优: $O(nc)$

最差: $O(nc)$

平均: $O(nc)$

贪心算法

贪心过程

伪代码

时间复杂度

活动安排

贪心过程

由于活动占用资源的时间没有限制，因此，后一种贪心选择更为合理。

为了在每一次贪心选择时快速查找具有最早结束时间的相容活动，先把 n 个活动按结束时间**非减序**排列。这样，贪心选择时取当前活动集合中结束时间最早的活动就归结为取当前活动集合中排在最前面的活动。

伪代码

```
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

各活动的起始时间和结束时间存储于数组s和f中且按结束时间的非减序排列

时间复杂度

$O(n)$

背包问题

贪心过程

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值 V_i/W_i ，然后，依贪心选择策略，将尽可能多的**单位重量价值最高**的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过 C ，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

伪代码

```
void Knapsack(int n,float M,float v[],float w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++) x[i]=0;
    float c=M;
    for (i=1;i<=n;i++)
    {
        if (w[i]>c) break;
        x[i]=1;
        c-=w[i];
    }
    if (i<=n) x[i]=c/w[i];
}
```

算法

knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为 $O(n\log n)$ 。

时间复杂度

$O(n\log n)$

最优装载问题

贪心过程

算法描述

最优装载问题可用贪心算法求解。**采用重量最轻者先装的贪心选择策略**，可产生最优装载问题的最优解。

伪代码

```
void Loading(int x[], Type w[], Type c, int n)
{
    int *t = new int [n+1];
    Sort(w, t, n);
    for (int i = 1; i <= n; i++) x[i] = 0;
    for (int i = 1; i <= n && w[t[i]] <= c; i++)
    {
        x[t[i]] = 1;
        c -= w[t[i]];
    }
}
```

时间复杂度

$O(n \log n)$

prim

```
1 int prim() {
2     int res = 0;
3     for (int i = 2; i <= V; i++) dis[i] = min(dis[i], G[1][i]);
4     for (int i = 2; i <= V; i++) {
5         int t = -1;
6         int temp = inf;
7         for (int j = 2; j <= V; j++) {
8             if (!vis[j] && dis[j] < temp) {
9                 t = j;
10                temp = dis[j];
11            }
12        }
13        if (t == -1) return -1;
14
15        res += temp;
16        vis[t] = true;
17        for (int j = 2; j <= V; j++) dis[j] = min(dis[j], G[t][j]);
18    }
19    return res;
20 }
```

kruska

```
1 int kruskal() {
2     int ans = 0;
3     int cnt = V;
4     for (int i = 1; i <= E; i++) {
5         int x = find(G[i].l), y = find(G[i].r);
```

```

6         if (x != y) {
7             fa[x] = y;
8             ans += G[i].w;
9             cnt--;
10        }
11        if (cnt == 1) break;
12    }
13    if (cnt != 1) return -1;
14    else return ans;
15 }

```

单源最短路径

贪心过程

其**基本思想**是，设置顶点集合S并不断地作**贪心选择**来扩充这个集合。一个顶点属于集合S当且仅当从源到该顶点的最短路径长度已知。

初始时，S中仅含有源。设u是G的某一个顶点，把从源到u且中间只经过S中顶点的路称为从源到u的特殊路径，并用数组dist记录当前每个顶点所对应的最短特殊路径长度。Dijkstra算法每次从V-S中取出具有最短特殊路径长度的顶点u，将u添加到S中，同时对数组dist作必要的修改。一旦S包含了所有V中顶点，dist就记录了从源到所有其它顶点之间的最短路径长度。

伪代码

```

1  int dijkstra() {
2      for (int i = 0; i < v - 1; i++) {
3          int node = -1;
4          for (int j = 1; j <= v; j++) {
5              if (!vis[j] && (node == -1 || dis[j] < dis[node])) {
6                  node = j;
7              }
8          }
9
10         for (int j = 1; j <= v; j++) {
11             dis[j] = min(dis[j], dis[node] + Graph[node][j]);
12         }
13
14         vis[node] = true;
15     }

```

```
16     if (dis[v] == inf) return -1;
17     else return dis[v];
18 }
```

时间复杂度

$O(n^2)$

哈夫曼编码

贪心过程

选两个最小点合成一个超级点

伪代码

```
1 void Huffman(PriorityQueue Q[ ], int n)
2 {
3     //Let Q be the priority queue.
4     Q = {initialize priority queue with frequencies of all symbol or message}
5     for (i = 1; i < n; i++)
6     {
7         create a new node z;
8         /* be greedy here */
9         x = extract_min(Q);
10        y = extract_min(Q);
11        Frequency(z) = Frequency(x) +Frequency(y);
12        /* weight of a tree = sum of the frequencies of its leaves */
13        Insert(z, Q);
14    }
15 }
16
```

时间复杂度

$T=O(n\log n)$

回溯

解向量

解空间树

限制函数

伪代码

时间复杂度

轮船装载问题

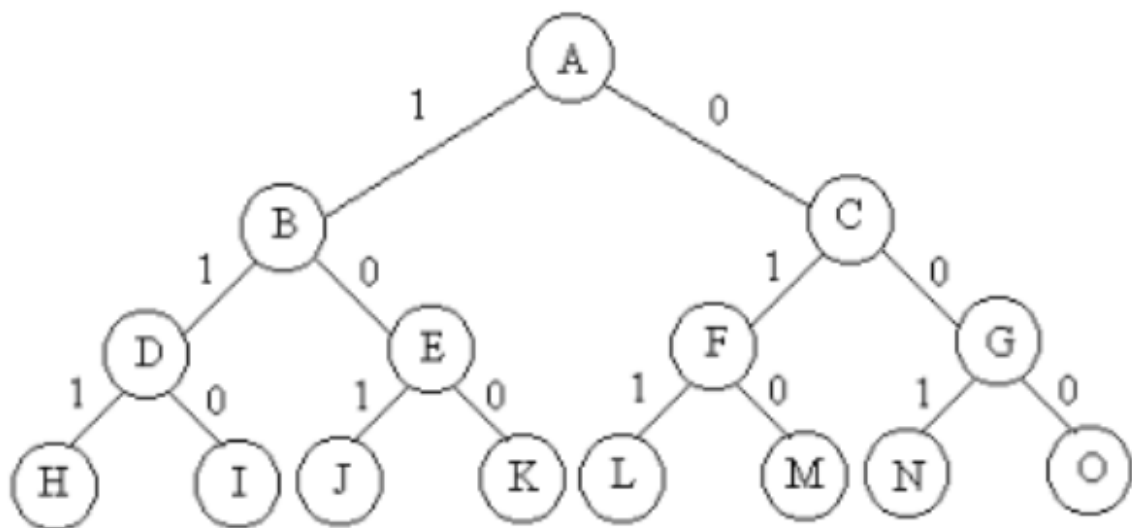
解向量

限制函数

$$\text{s.t. } \sum_{i=1}^n w_i x_i \leq c_1$$

$$x_i \in \{0,1\}, 1 \leq i \leq n$$

解空间树



伪代码

```

1 void loading(int i)
2 {
3     if (i > n)
4     {
5         for (int j = 1; j <= n; j++)
6             cout << x[j] << ' ';
7         cout << endl;
8         if (cw > bestw)
9         {
10            for (int j = 1; j <= n; j++)
11                bestx[j] = x[j];
12            bestw = cw;
13        }
14    }
15 }

```

```

14     return;
15 }
16 if (cw + w[i] <= c)
17 {
18     x[i] = 1;
19     cw += w[i];
20     loading(i + 1);
21     x[i] = 0; //无论进入下一层与否都会执行，这是更保险的位置
22     cw -= w[i];
23 }
24     loading(i + 1);
25 }
26

```

时间复杂度

$O(2^n)$

01背包

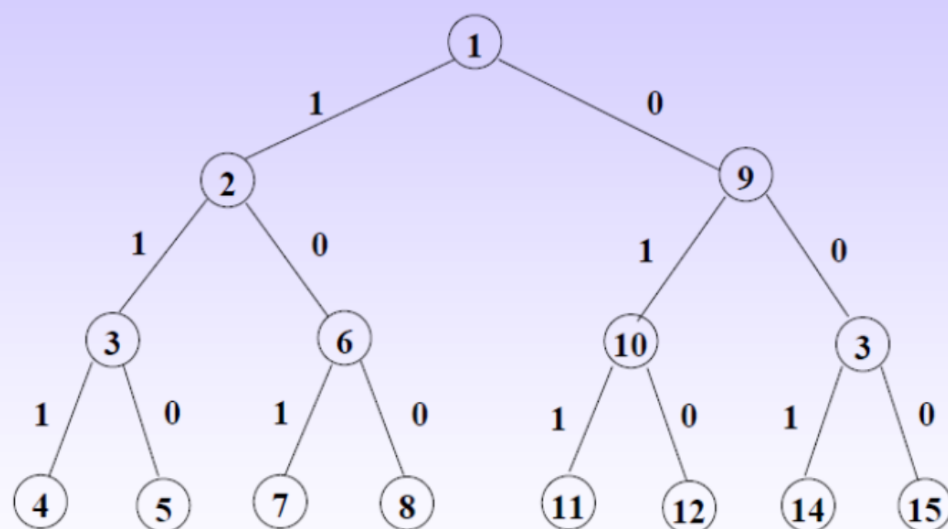
解向量

(2) 可能解由一个等长向量 $\{x_1, x_2, \dots, x_n\}$ 组成，其中 $x_i=1(1 \leq i \leq n)$ 表示物品 i 装入背包， $x_i=0$ 表示物品 i 没有装入背包，当 $n=3$ 时，其解空间是：

$\{(0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}$

解空间树

对于 $n=3$ 的0/1背包问题，其解空间树如下图所示，树中的8个叶子结点分别代表该问题的8个可能解。



对物品1的选择

对物品2的选择

对物品3的选择

限制函数

$$\sum_{i=1}^n w_i x_i \leq c$$

伪代码

```
Typep Knap<Typew, Typep>::Bound(int i)
{ // 计算上界
    Typew cleft = c - cw; // 剩余容量
    Typep b = cp;
    // 以物品单位重量价值递减序装入物品
    while (i <= n && w[i] <= cleft) {
        cleft -= w[i];
        b += p[i];
        i++;
    }
    // 装满背包
    if (i <= n) b += p[i]/w[i] * cleft;
    return b;
}
```

```

template <class Typew, class Typep>
void Knap < Typew, Typep> :: Backtrack(int i)
{
    if (i > n) //到达叶子节点
    {
        bestp = cp;
        return;
    }
    if ((cw + w[i] <= c)
    {
        cw += w[i];
        cp += p[i];
        Backtrack(i + 1);
        cw -= w[i];
        cp -= p[i];
    }
    if (Bound ( i + 1) > bestp)
        Backtrack( i + 1 );
}

```

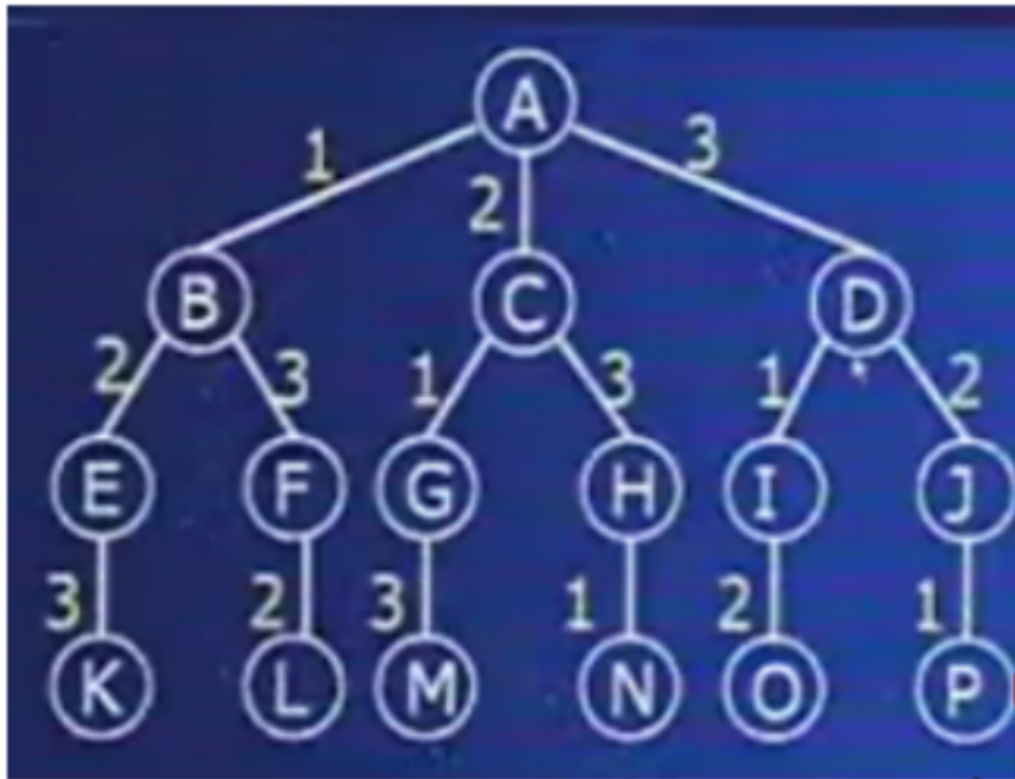
时间复杂度

$O(2^n)$

作业调度

解向量

解空间树



限制函数

伪代码

批处理作业调度问题

```
void Flowshop::Backtrack(int i)
```

```
{
    if (i > n) {
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestf = f;
    }
    else
        for (int j = i; j <= n; j++) {
            f1+=M[x[j]][1];
            f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+M[x[j]][2];
            f+=f2[i];
            if (f < bestf) {
                Swap(x[i], x[j]);
                Backtrack(i+1);
                Swap(x[i], x[j]);
            }
            f1-=M[x[j]][1];
            f-=f2[i];
        }
}
```

北京邮电大学

t_{ji}	机器
作业1	2
作业2	3
作业3	2



时间复杂度

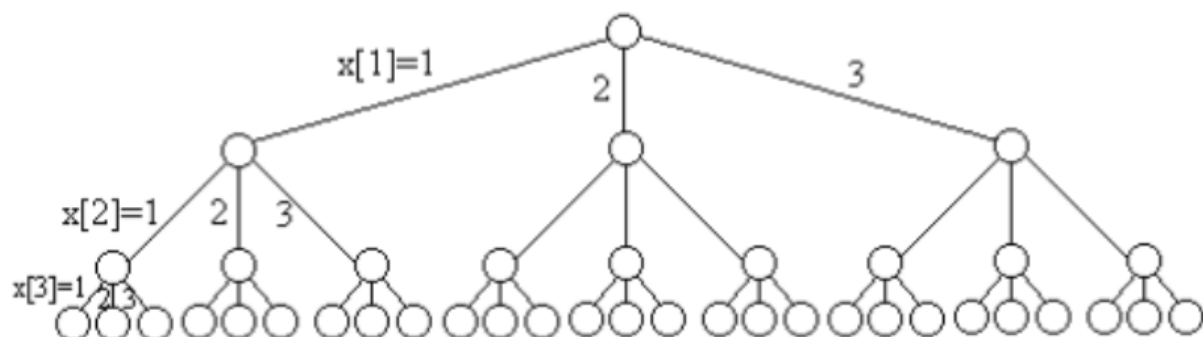
$O(n!)$

图的M着色

解向量

- 解向量: $(x_1, x_2, \dots, x_n)$ 表示顶点 i 所着颜色 $x[i]$

解空间树



限制函数

伪代码

```
void Color::Backtrack(int t)
```

```
{
    if (t>n) {
        sum++;
        for (int i=1; i<=n; i++)
            cout << x[i] << ' ';
        cout << endl;
    }
    else
        for (int i=1; i<=m; i++) {
            x[t]=i;
            if (Ok(t)) Backtrack(t+1);
        }
}
```

```
bool Color::Ok(int k)
```

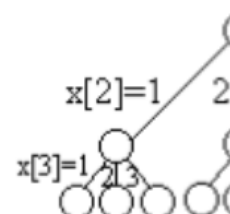
```
{// 检查颜色可用性
```

```
    for (int j=1; j<=n; j++)
```

```
        if ((a[k][j]==1) && (x[j]==x[k])) return false;
```

```
    return true;
```

```
}
```



时间复杂度

复杂度分析

图m可着色问题的解空间树中内结点个数是 $\sum_{i=0}^{n-1} m^i$
 对于每一个内结点，在最坏情况下，用ok检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时 $O(mn)$ 。因此，回溯法总的时间耗费是

$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1) / (m - 1) = O(nm^n)$$

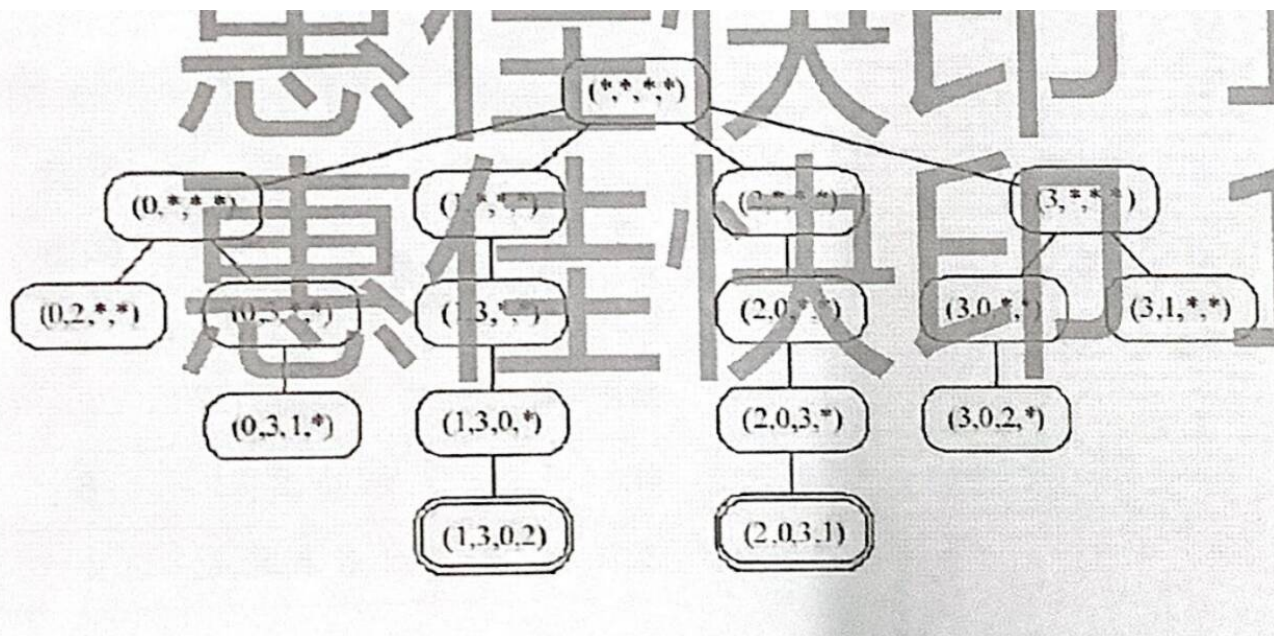
$O(nm^n)$

N皇后

解向量

解空间树

排列树



限制函数

$$x_i \neq x_j$$

$$|i - x_i| \neq |j - x_j|$$

伪代码

```
1  bool is_ok(int row)
2  {
3      for (int j = 0; j != row; j++)
4      {
5          if (c[row] == c[j] || row - c[row] == j - c[j] || row + c[row] == j + c[j])//
          保证不在同一列&保证不在同一斜线
6              return false;
7      }
8      return true;
9  }
10 void queen(int row){
11     if (row == n)
12     {
13         total++;
14         for (int j = 0; j < n; j++)
15             cout << c[j] << ' ';
16     }
17     else
18         for (int col = 0; col < n; col++)
19         {
20             c[row] = col;//row+1的递归过程已经保证不在同一排
21             if (is_ok(row))
22                 queen(row + 1);
23         }
24 }
25
```

时间复杂度

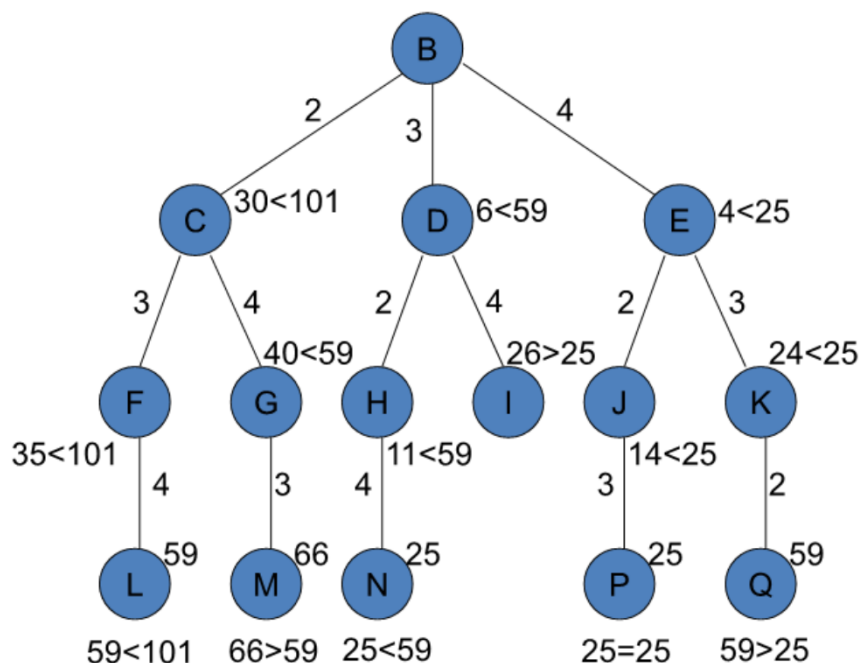
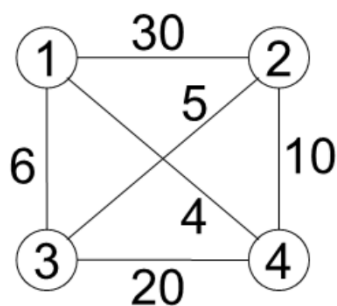
$O(n!)$

旅行商问题

解向量

解空间树

带下界函数:



限制函数

剪枝条件/约束1: 如果当前正在考虑的顶点j与当前路径中的末端结点i没有边相连, 即 $w[i, j] = \infty$, 则不必搜索j所在分支

如果 $B(i) \geq \text{bestw}$, 则停止搜索x[i]分支及其下面的层,

其中, bestw代表到目前为止,在前面的搜索中,从其它已经搜索过的路

伪代码

```

1  bool inx(int i)//判断当前结点i是否被访问过
2  {
3      for (int j = 0; j <= i - 1; j++)
4          if (i == x[j]) return true;
5      return false;
6  }
7  void TSPrecursion(int i)//回溯法计算
8  {
9      int u;
10     if (i == n) // 结束
11     {
12         cv = cv + w[x[n - 1]][1];
13         if (cv <= v_best) // 小于最短路径
14         {
15             v_best = cv;// 更新最优解
16         }
17         for (int j = 0; j <= n - 1; j++)
18         {
19             x_best[j] = x[j];
20             cout << x_best[j]<<' ';

```

```

21     }
22     cv = cv - w[x[n - 1]][1]; //回溯 (最大值回溯)
23     return;
24 }
25 for (u = 2; u <= n; u++)
26 {
27     if (inx(u) == false && w[u][x[i - 1]] != -1) // 剩余可选城市
28     {
29         if (cv + w[x[i - 1]][u] <= v_best) // 加入后小于最短路径
30         {
31             x[i] = u; // 加入
32             cv = cv + w[x[i - 1]][u]; // 更新当前解
33             TSPrecursion(i + 1); // 搜索下一层
34             x[i] = 0; // 回溯层数
35             //因为需要回头来判断当前结点是否被访问过，所以需要回溯
36             cv = cv - w[x[i - 1]][u]; //回溯路径长度
37         }
38     }
39 }
40 }
41

```

时间复杂度

$O(n!)$

一些证明方法

最优子结构：原问题的最优解包含子问题的最优解。

画重点了！！！！下边是证明：

设 $(y_1, y_2, \dots, y_n)$ 是 $(*)$ 的一个最优解，则 $(y_2, y_3, \dots, y_n)$ 是下面子问题的最优解：

$$\begin{aligned} & \max \sum_{i=2}^n v_i x_i \\ & \text{s.t.} \begin{cases} \sum_{i=2}^n w_i x_i \leq C - w_1 y_1 \\ x_i \in \{0, 1\}, 2 \leq i \leq n \end{cases} \end{aligned}$$

证明：（反证法）假设 $(z_2, z_3, \dots, z_n)$ 是上述子问题的最优解，即 $(y_2, y_3, \dots, y_n)$ 不是子问题的最优解——不存在最优子结构则：

$$\begin{aligned} \sum_{i=2}^n v_i y_i & \leq \sum_{i=2}^n v_i z_i \\ w_1 y_1 + \sum_{i=2}^n w_i z_i & \leq C \end{aligned}$$

则有

$$\sum_{i=1}^n v_i y_i \leq \sum_{i=2}^n v_i z_i + v_1 y_1$$

说明 $(y_1, z_2, z_3, \dots, z_n)$ 是比 $(y_1, y_2, y_3, \dots, y_n)$ 更优的一个解，与原假设矛盾！

1.证明：最短路径问题满足最优子结构。

设 P_{uv} 是从顶点 u 到顶点 v 的最短路径， w 是路径内部异于 u 和 v 的任一点，推断 P_{uw} 和 P_{wv} 也分别是 u 到 w 和从 w 到 v 的最短路径。

下面用反证法证明上面的推断成立。

1) 若存在路径 P'_{uw} 比 P_{uw} 更短，并且 P'_{uw} 和 P_{wv} 无关，即他们不含有除了 w 之外的公共点。则利用[复制粘贴](#)法用 P'_{uw} 代替 P_{uw} ，这样从 u 到 v 就存在更短的路径，这与 P_{uv} 是 u 到 v 的最短路径这一假设矛盾，所以从 u 到 w 的所有和 P_{wv} 无关的路径中， P_{uw} 最短

2) 若存在路径 P'_{uw} 比 P_{uw} 更短，并且 P'_{uw} 和 P_{wv} 有关，设存在公共点 x ，这时候不能用复制粘贴将 P'_{uw} 代替 P_{uw} 因为最短路径不允许存在环。但是可以将 P'_{uw} 分解为 P'_{ux} 和 P'_{xw} ， P_{wv} 分解为 P'_{wx} 和 P'_{xv} ，

$$P'_{uv} = P'_{ux} + P'_{xv}$$

由于 P'_{uw} 比 P_{uw} 更短，与 P_{uv} 是最短路径矛盾。所以从 w 到 v 的所有和 P_{wv} 有关的路径中， P_{wv} 最短。

综合1)和2)可知 P_{uw} 是子问题 u 到 w 的最短路径。对称的可证明 P_{wv} 也是顶点 w 到 v 的最短路径。所以最短路径问题满足最优子结构。

2、贪心算法^Q的证明方式

(1) 替换法（反证法）

- 先依靠贪心算法得出最优解，再证明：将任意步骤的决策替换为任意的其他选择，最后结果都不可能达到更优。

eg: 背包负重问题

题目：给定数组 arr 表示 n 个物品各自的重量，背包负重上限为 N ，要求在不超过负重上限的情况下，尽可能的多带物品。

解法：贪心算法，每次总拿取最轻物品放入背包

替换法证明：设利用贪心算法得出的最优解为 $[n_1, n_2, n_3, \dots, n_x]$ ，现用 m 替换 n_1 (n_1 已是“当前最轻”，故 $m > n_1$)，证明能否得到更优解 $[m, n_2, n_3, \dots, n_x, n_y]$ (比原最优解多放入一个物品 n_y)

观察可知：“更优解前 x 项之和”必定大于“原最优解 x 项之和”，且更优解还多了一项 n_y 都没有超限，说明原最优解加上 n_y 也必定不会超限，原最优解应为 $[n_1, n_2, n_3, \dots, n_x, n_y]$ 与题设相悖，故反证错误，不存在这样的替换方法。**即：替换最优解任意项后，都无法达到更优，得证。**

- 替换法实质上是在尝试证明“局部最优”与“整体最优”的关联性，若一旦失去局部最优，也必将失去整体最优，则说明贪心策略正确可行。